

Retrieval-Augmented Robotic Manipulation with Pretrained 3D Representations

Abstract

This project studies whether geometric similarity between robotic manipulation scenes provides a useful retrieval signal for reusing past experiences. We build an RLBench experience database and compare random, pose, image/color, geometric, Uni3D, and Point Transformer V3 retrieval. The primary evaluation contains 19 tasks and 1,804 episodes. We also evaluate robustness to viewpoint, occlusion, and geometry noise, and test a frozen-backbone trajectory-state projection head with a held-out test-to-train protocol. The results show that 3D retrieval is meaningfully better than random retrieval, but color and appearance baselines are stronger in this dataset. Uni3D is stronger than PTV3 in the main results. The projection head does not improve held-out performance over original Uni3D. The simulator experiment validates integration but is not a learned planning benchmark because the exported episodes do not contain explicit action arrays.

The complete implementation, reproduction instructions, and supporting scripts are available in the project repository: [RAG for Robotics on GitHub](#).

1. Introduction and Related Work

1.1 Motivation and Research Question

Retrieval-augmented robotics uses a memory of previous robot experiences to support decisions in a new scene. Each memory item can be represented as a tuple containing a scene observation, a trajectory, and outcome metadata. Given a new point-cloud observation, the system retrieves nearby experiences and can pass their trajectories to a downstream policy or planner.

The research question is:

<i>Does 3D geometric similarity between scenes, encoded via a pretrained 3D</i>
<i>representation, provide a meaningful retrieval signal for robotic</i>
<i>manipulation planning, and under what scene variations does this signal remain</i>
<i>reliable?</i>

The initial hypothesis was that geometric representations would be more robust than appearance-based representations under viewpoint, occlusion, and object variation. The project therefore treats representation quality and similarity as the primary experimental variables. A learned action-generating planner was part of the proposed long-term direction, but the completed implementation focuses on retrieval and offline downstream evidence.

1.2 Related Work

RLBench was introduced by James et al. as a robot-learning benchmark and simulation environment containing diverse manipulation tasks and demonstrations. It is the source of the simulated manipulation episodes used here. The present work differs from a policy-learning benchmark because the main metric evaluates whether scene embeddings retrieve episodes from the same task group.

Mandlekar et al. introduced robomimic as a reproducible framework for learning from offline robot demonstrations and showed that demonstration quality and algorithmic choices strongly affect manipulation learning. This motivates our explicit manifest, fixed splits, and artifact-based evaluation. Our focus is earlier in the pipeline: selecting useful demonstrations before training a control policy.

MimicGen studies scalable generation of robot-learning demonstrations by adapting source demonstrations to new contexts. This is relevant future work for our system, because retrieved trajectories could eventually be transformed before execution. Our current system retrieves trajectories but does not adapt them into new executable plans.

Kuroki et al. present retrieval-augmented policy training using a database of cooperative robot behaviors. Their work is a close conceptual precedent for using memory retrieval to support robot control. Our contribution is narrower: we isolate the question of whether pretrained 3D scene representations are useful retrieval keys, without claiming their policy-training or real-robot success setting.

Liu et al. study policies augmented with motion-planner supervision. This work highlights the distinction between a retrieval module and an action-generating policy. Our simulator pilot validates execution infrastructure and replay only; it does not implement a learned policy of this kind.

Uni3D, by Zhou et al., is a pretrained 3D foundation model that aligns point cloud features with image-text aligned representations. We use an official Uni3D checkpoint as a learned 3D retrieval encoder, while evaluating it on robotic manipulation episodes rather than its original object-level benchmarks.

Point Transformer V3, by Wu et al., emphasizes scalable serialized point-cloud processing and efficient large receptive fields. We use the Pointcept v1.5.2 runtime and a compatible pretrained checkpoint as a second modern 3D encoder. The comparison tests whether a modern point-cloud backbone automatically gives the best task-retrieval signal. The results suggest that this depends on the dataset and the task-relevance definition.

1.3 Positioning

The project combines a robotic experience database, pretrained 3D embeddings, nearest-neighbor retrieval, controlled perturbation tests, and an offline trajectory-transfer proxy. It does not claim to deliver a complete learned planner. This distinction is important because retrieval quality and planning success are related but different measurements.

1.4 Proposal Scope and Deliverables

The approved proposal asked whether pretrained 3D geometric representations can retrieve useful manipulation experiences, whether they remain reliable under scene variations, and whether retrieved experiences can support downstream planning. The completed scope delivers the first two parts rigorously and provides offline downstream and simulator-integration evidence for the third. The project deliberately reports the learned-planner component as future work rather than presenting replay as a learned planning result.

2. Method

2.1 Dataset and Experience Database

The full exported dataset contains 19 RLBench tasks and 1,804 episodes. The task set is: reach_target, open_drawer, slide_block_to_color_target, sweep_to_dustpan_of_size, meat_off_grill, turn_tap, put_item_in_drawer, close_jar, reach_and_drag, stack_blocks, light_bulb_in, put_money_in_safe, place_wine_at_rack_location, put_groceries_in_cupboard, place_shape_in_shape_sorter, push_buttons, insert_onto_square_peg, stack_cups, and place_cups.

Each episode has a manifest record, observation arrays, trajectory data, task identity, and metadata. Relevance annotations are task-group based: an episode is relevant when it belongs to the same task group as the query. This is a well-defined retrieval protocol, but it is not equivalent to measuring whether the retrieved trajectory is executable in a new scene.

2.1.1 Code Structure

The repository is organized around a small, reproducible experiment pipeline:

```
RAG_for_Robotics/<br>|-- src/action_retrieval/<br>| |-- data/ Dataset schema, export, transforms, validation<br>| |-- retrieval/ Episode loading, encoders, embeddings, ranking<br>| |-- evaluation/ Retrieval metrics and evaluation protocol<br>| |-- simulation/ RLBench source import and trajectory utilities<br>| |-- downstream/ Offline downstream transfer interfaces<br>| |-- visualization/ Visualization package namespace<br>| -- utils/ Environment and reproducibility helpers<br>|-- scripts/<br>| -- evaluate_retrieval.py Main retrieval evaluation<br>| -- evaluate_robustness.py Viewpoint/occlusion/noise tests<br>| -- train_action_aware_projection.py Frozen-backbone projection head<br>| -- evaluate_projected_embeddings.py Held-out projection evaluation<br>| -- evaluate_downstream_proxy.py Offline trajectory-transfer proxy<br>| -- run_rlbench_planning_pilot.py Guarded simulator replay pilot<br>| -- plot_report_figures.py Report figure generation<br>|-- configs/ Dataset, encoder, retrieval, and experiment settings<br>|-- slurm/ Cluster jobs for setup, evaluation, and diagnostics<br>|-- tests/ Unit and integration tests for project code<br>|-- third_party/ External RLBench/PyRep integration boundary<br>|-- requirements.txt General Python dependencies<br>|-- requirements-cluster.txt Validated Cluster additions and versions<br>|-- ENVIRONMENT_REPRODUCTION.md Exact environment and checkpoint
```

The `src/` package contains reusable project logic, `scripts/` contains experiment entry points, `configs/` makes settings explicit, and `slurm/` encodes the Cluster execution environment. Large datasets, model checkpoints, and generated result archives are kept outside the Git source tree and are referenced by their recorded paths and hashes.

2.2 Retrieval Encoders

Method	Representation
---	---
random	Random candidate ranking
pose_descriptor	Compact pose and state descriptor
rgb_histogram	Global RGB histogram
global_color	Global color statistics
geometry_only	Point-cloud geometry descriptor without color
uni3d	Official pretrained Uni3D point-cloud encoder
ptv3	Official Pointcept PTv3 runtime and pretrained checkpoint

All methods use the same episode database, relevance annotations, and ranking metrics. The retrieval cutoffs are $k=1$, 2, and 3.

2.3 Metrics

For each query, candidates are ranked by embedding similarity. We report precision@k , recall@k , mean reciprocal rank (MRR), mean average precision@k (MAP@k), normalized discounted cumulative gain (NDCG@k), Top-1 accuracy, mean similarity scores, median first relevant rank, and hit rate@k .

2.4 Action-Aware Projection Head

The projection-head pilot freezes the Uni3D backbone and trains a small MLP to predict a compact trajectory-state signature derived from gripper pose. The MLP is not an action-generating planner and does not fine-tune Uni3D. Training uses 422 episodes, while held-out evaluation uses 141 test queries against 422 train candidates, with zero train/test overlap.

2.5 Software Environment

The experiments required a larger native and scientific Python stack than the retrieval scripts alone. Important components were PyTorch with the validated CUDA build, torchvision, torch-geometric, torch-scatter, torch-sparse, torch-cluster, spconv-cu124, cumm, timm, open3d, pyarrow, fastparquet, scipy, scikit-learn, matplotlib, and easydict. The simulator pilot also required gymnasium, cffi, natsort, PyRep, RLBench, and CoppeliaSim. These dependencies are separated between `requirements.txt` and `requirements-cluster.txt`; exact installed versions and checkpoint hashes are recorded in the reproducibility artifacts.

2.6 Robustness Protocol

Robustness perturbations are applied to query observations while the candidate database remains clean. The tested conditions are viewpoint rotation, partial occlusion, and geometry noise. This is a controlled proxy for scene variation; it does not replace evaluation on independently generated object geometries.

2.7 Implementation Ownership and External Code

The experiment orchestration and evaluation code in this repository was written for this project. This includes dataset export and validation, manifest and annotation handling, retrieval encoders and ranking, evaluation metrics, robustness perturbations, projection-head training/evaluation, reproducibility scripts, SLURM wrappers, and the simulator pilot harness.

Uni3D and Pointcept/PTv3 are external research codebases used as model runtimes. Their pretrained checkpoints and native dependencies were not written as part of this project. They are loaded through adapters in this repository, with configuration, checkpoint paths, key remapping, device selection, and fallback handling implemented here. RLBench, PyRep, CoppeliaSim, PyTorch, PyG, sponconv, cumm, and other packages are also external dependencies. Their licenses and upstream references should be preserved in the submitted code folder.

The scientific contribution of this implementation is the common retrieval interface, controlled comparison protocol, task-group relevance evaluation, robustness setup, held-out projection-head experiment, and analysis of the resulting evidence. We do not claim authorship of the upstream model architectures or pretrained weights.

3. Evaluation and Results

3.1 Full 19-Task Retrieval

The primary results are stored in `retrieval_full/v2_multitask_full/summary_metrics.csv` and `retrieval_full/v2_multitask_full/evaluation.json`.

| Method | Top-1 accuracy | Hit rate@3 | MRR@3 |

| --- | ---: | ---: | ---: |

| random | 0.0615 | 0.1558 | 0.1003 |

| pose_descriptor | 0.9462 | 0.9734 | 0.9587 |

| rgb_histogram | 0.9390 | 0.9734 | 0.9542 |

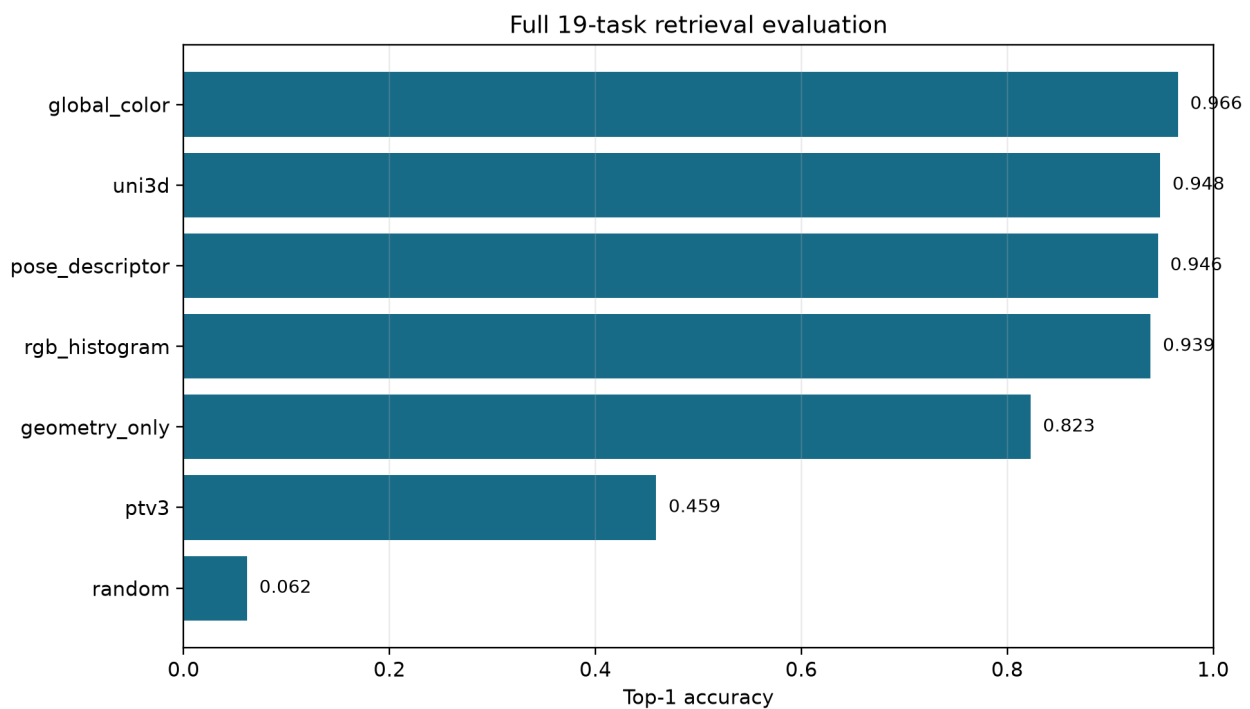
| global_color | 0.9656 | 0.9856 | 0.9747 |

| geometry_only | 0.8226 | 0.8830 | 0.8492 |

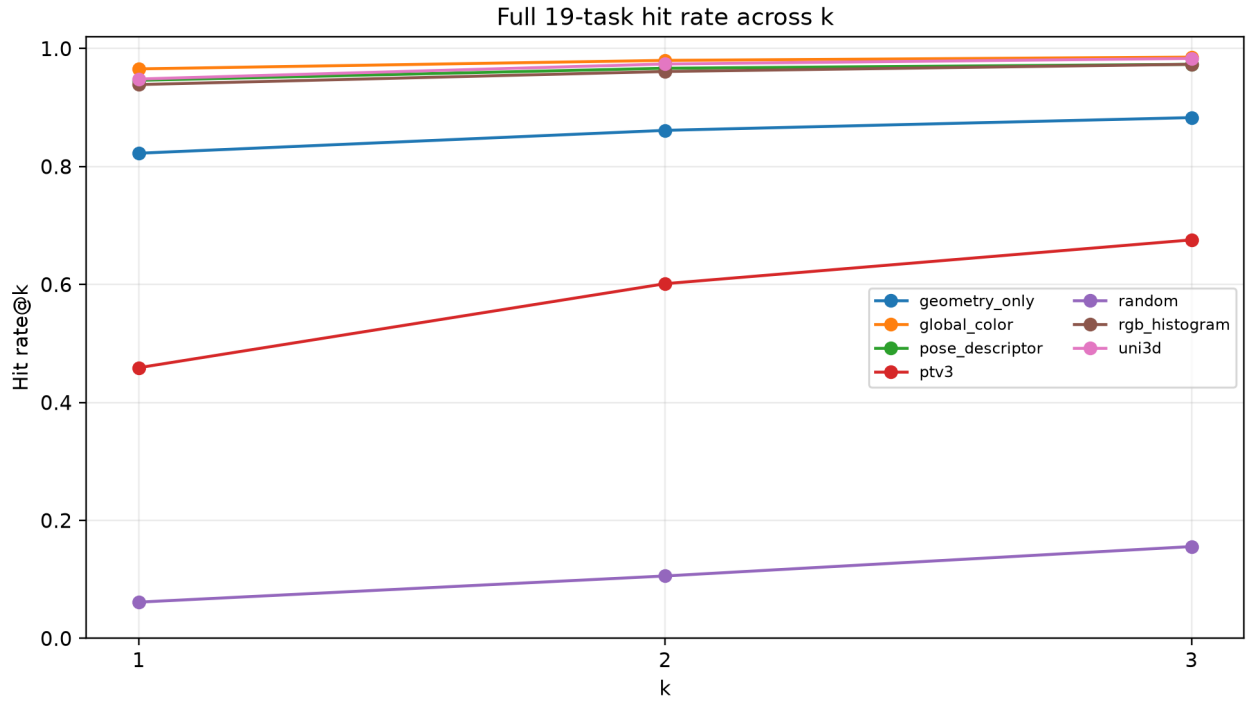
| uni3d | 0.9484 | 0.9834 | 0.9643 |

| ptv3 | 0.4590 | 0.6757 | 0.5550 |

The full-dataset result shows that all non-random methods provide a meaningful task retrieval signal. Global color is strongest by Top-1 accuracy, followed by Uni3D and pose_descriptor. Geometry-only retrieval is substantially above random but below the color-aware methods. PTv3 is above random but weaker than the other tested representations in this configuration.



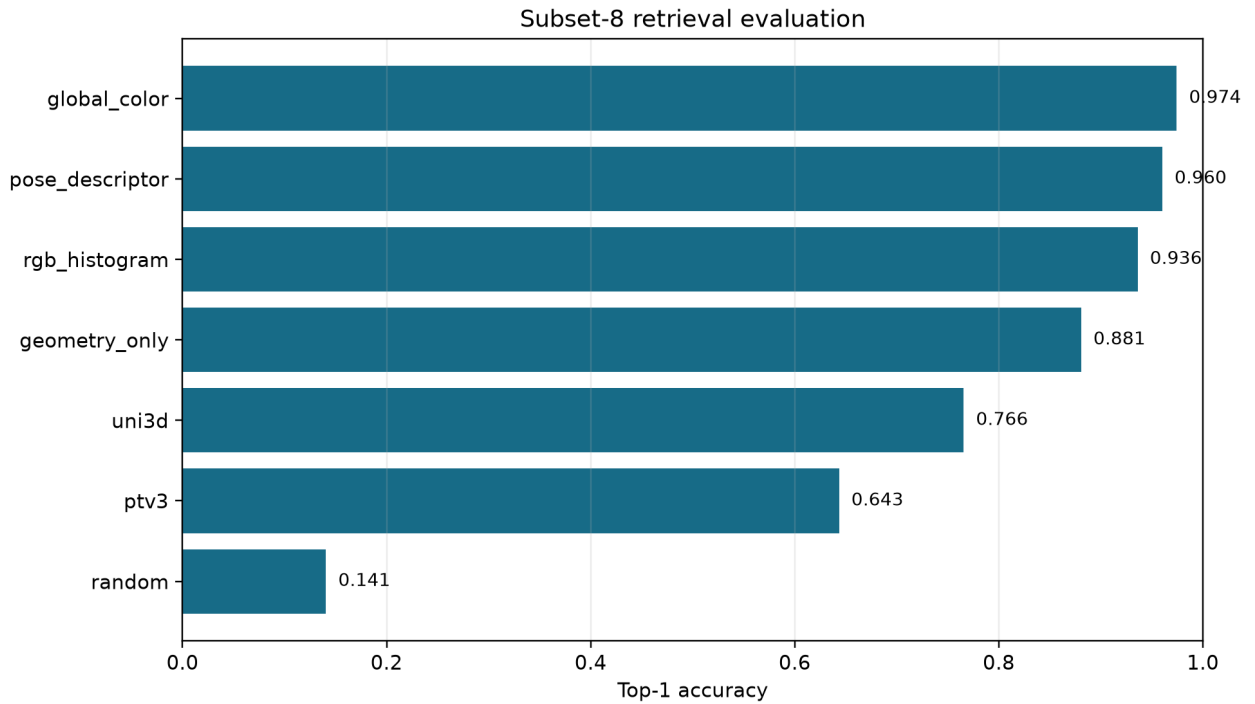
Full Top-1 comparison



Full hit rate by k

3.2 Subset-8 Retrieval

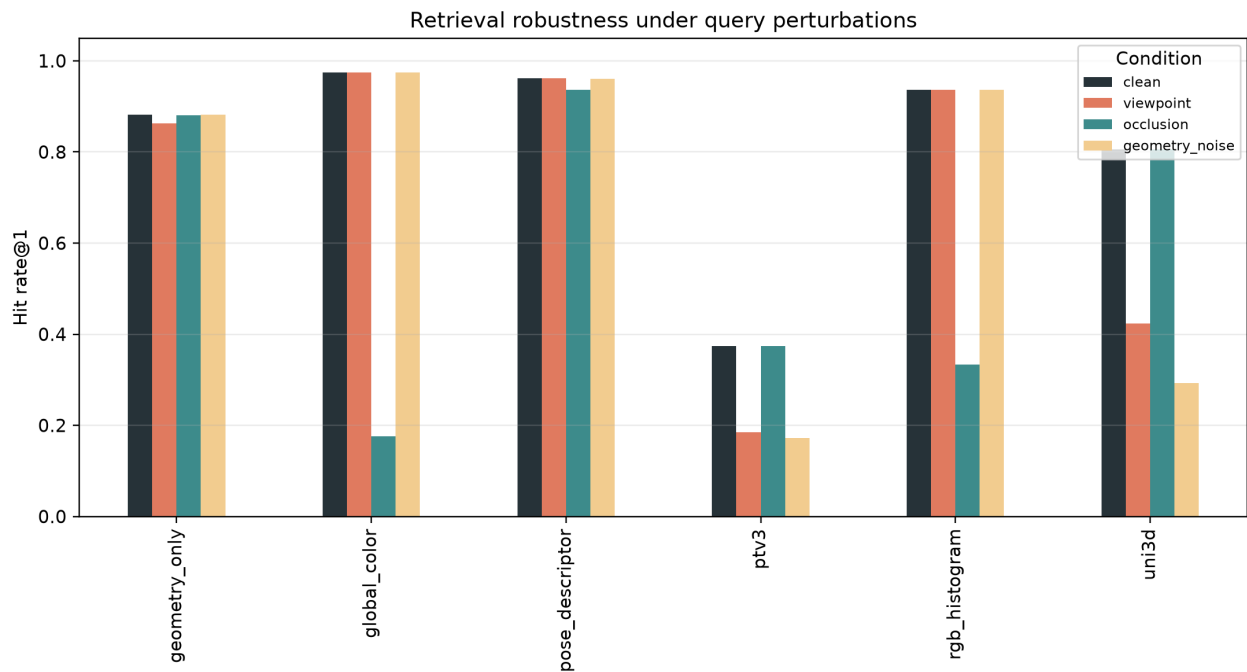
The focused subset contains 704 episodes across eight tasks. The subset results are stored in `retrieval_all/v2_multitask_subset8`. On this subset, the Top-1 accuracies were 0.9744 for `global_color`, 0.9602 for `pose_descriptor`, 0.9361 for `rgb_histogram`, 0.8807 for `geometry_only`, 0.7656 for `Uni3D`, 0.6435 for `PTv3`, and 0.1406 for `random`. The ordering reinforces the conclusion that appearance and task-correlated color are strong signals in this benchmark.



Subset-8 Top-1 comparison

3.3 Robustness

The robustness evaluation is stored in `robustness/v2_multitask_subset8/summary_metrics.csv` and `robustness/v2_multitask_subset8/evaluation.json`. It evaluates all six non-random retrieval representations under viewpoint, occlusion, and geometry noise perturbations. Since the perturbations affect queries only, the results measure sensitivity of the query embedding and ranking function rather than a change in the database.



Robustness comparison

The robustness experiment is evidence about controlled perturbation behavior, not proof of generalization to every possible object geometry or real-world occlusion pattern.

3.4 Held-Out Projection Head

The held-out comparison uses the test split as queries and the train split as candidates. The split contains 422 train episodes and 141 test episodes, with zero overlap.

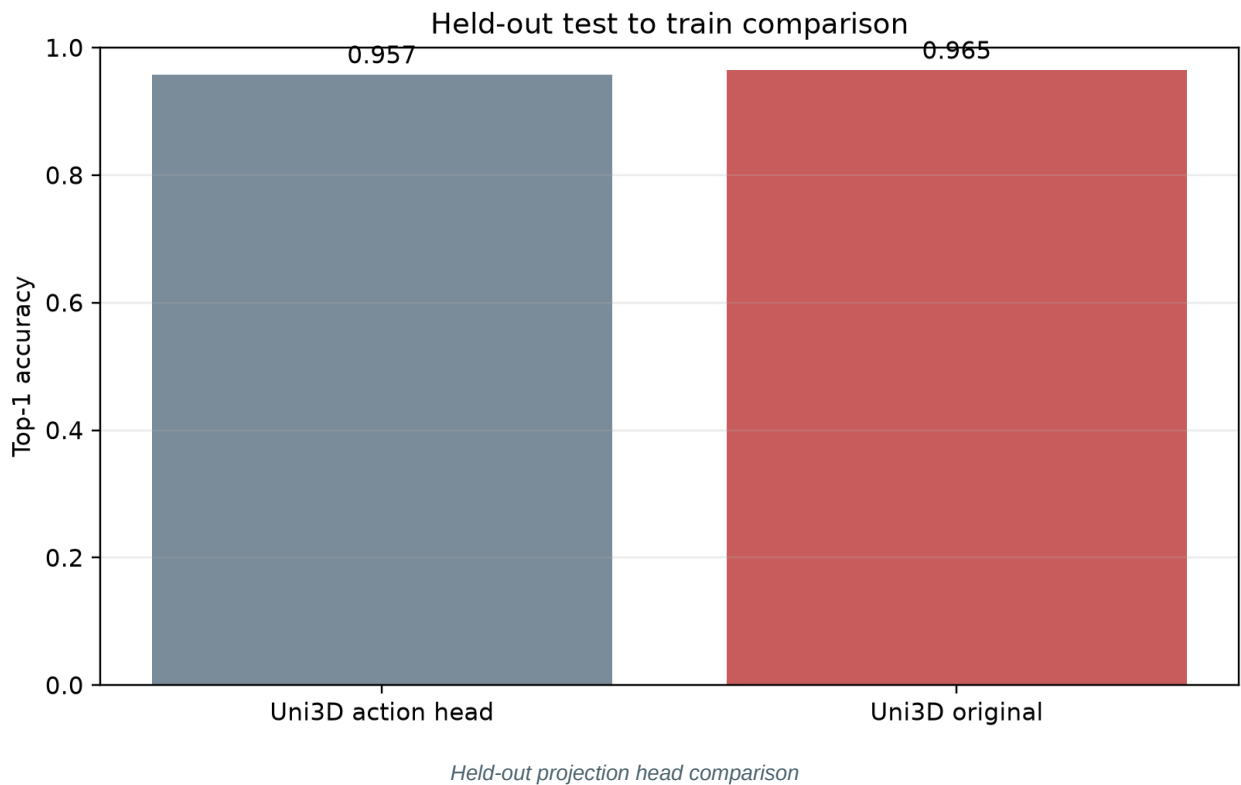
| Method | Top-1 accuracy | Precision@2 | Hit rate@2 |

| --- | ---: | ---: | ---: |

| Uni3D original | 0.9645 | 0.9433 | 0.9858 |

| Uni3D action head | 0.9574 | 0.9716 | 0.9929 |

The action head is slightly worse at Top-1 on held-out data, although it has a higher Precision@2 and Hit rate@2. The earlier 0.9787 Top-1 value was obtained when evaluating all 704 episodes and is therefore reported only as an in-sample diagnostic, not as the primary generalization result.



3.5 Offline Downstream Proxy

The downstream experiment in `downstream/v2_multitask_full` evaluates retrieved trajectory transfer offline. It is useful as a bridge from retrieval to downstream use, but it does not execute a learned policy in the simulator.

3.6 Simulator Integration Pilot

The RLBench/CoppeliaSim pilot successfully initialized the simulator in headless mode and executed the replay harness. Four available ReachTarget episodes were checked. The stored trajectories contained derived joint-position sequences, not explicit ground-truth action arrays; the pilot therefore recorded 0/4 successes. This result diagnoses an action/state and initial-scene mismatch and must not be interpreted as evidence that Uni3D or PTV3 fails at learned planning.

4. Discussion and Conclusions

4.1 Main Findings

First, 3D geometric retrieval is useful: geometry-only, Uni3D, and PTV3 all perform above random retrieval. Second, the tested color and pose baselines are stronger than the learned 3D representations in this task-group evaluation. Third, Uni3D is stronger than PTV3 in both the full and subset results, but it does not dominate the color baselines. Fourth, the frozen trajectory-state projection head does not improve held-out Top-1 accuracy over original Uni3D.

These results provide a meaningful negative finding: a modern pretrained 3D backbone is not automatically the limiting or best retrieval signal when task identity is strongly correlated with appearance and color.

4.2 Limitations and Future Work

- Relevance is defined primarily by task-name groups, not by downstream action compatibility or measured execution success.
- The robustness conditions are synthetic query perturbations.
- The action-aware head predicts a trajectory-state signature, not action sequences.

- The simulator pilot uses derived joint-position replay because explicit action

arrays are absent.

- No Flow Matching planner or other learned action-generating policy was trained.
- Future work should collect explicit actions, train a trajectory/action decoder,

define action-aware relevance, and evaluate independently generated scene variations.

4.3 Conclusion

The project answers the central retrieval question positively but with an important qualification. Pretrained 3D representations provide a meaningful signal for retrieving manipulation experiences, yet the signal is not universally more robust or more accurate than simple appearance-aware baselines. Uni3D is the stronger of the two tested learned 3D encoders, while PTV3 provides a useful contrasting modern backbone. The completed evidence supports retrieval as a viable component of retrieval-augmented robotics and identifies representation-task alignment as the central open issue.

Appendix A: Machine-Readable Evidence

Selected JSON artifacts should accompany the PDF or final results archive. They support, rather than replace, the human-readable tables and figures:

- Full retrieval evaluation.json.
- Subset-8 retrieval evaluation.json.
- Robustness evaluation.json.
- Held-out Uni3D and action-head evaluation.json files.
- Simulator planning-pilot evaluation.json.
- Reproducibility environment.txt and checkpoint_sha256.txt.

Implementation and reproduction instructions are provided in README.md, ENVIRONMENT_REPRODUCTION.md, requirements.txt, and requirements-cluster.txt.

References

1. Stephen James et al. "RLBench: The Robot Learning Benchmark & Learning Environment." arXiv:1909.12271, 2019. <https://arxiv.org/abs/1909.12271>
2. Ajay Mandlekar et al. "What Matters in Learning from Offline Human Demonstrations for Robot Manipulation." arXiv:2108.03298, 2021. <https://arxiv.org/abs/2108.03298>
3. Ajay Mandlekar et al. "MimicGen: A Data Generation System for Scalable Robot Learning using Human Demonstrations." arXiv:2310.17596, 2023. <https://arxiv.org/abs/2310.17596>
4. So Kuroki et al. "Multi-Agent Behavior Retrieval: Retrieval-Augmented Policy Training for Cooperative Push Manipulation by Mobile Robots." arXiv:2312.02008, 2023. <https://arxiv.org/abs/2312.02008>
5. I-Chun Arthur Liu et al. "Distilling Motion Planner Augmented Policies into Visual Control Policies for Robot Manipulation." CoRL, 2022. <https://proceedings.mlr.press/v164/liu22b.html>
6. Junsheng Zhou et al. "Uni3D: Exploring Unified 3D Representation at Scale." arXiv:2310.06773, 2023. <https://arxiv.org/abs/2310.06773>
7. Xiaoyang Wu et al. "Point Transformer V3: Simpler, Faster, Stronger." CVPR,

2024, pp. 4840-4851. https://openaccess.thecvf.com/content/CVPR2024/html/Wu_Point_Transformer_V3_Simpler_Faster_Stronger_CVPR_2024_paper.html